NAME: TANKISO MASOEBE

ADVANCED DATABASE WK12

1.Hands-On Backup and Recovery Using PostgreSQL

1. Introduction

Database backup and recovery are important processes used to protect data against accidental deletion, system failures, and data corruption. PostgreSQL provides tools such as pg_dump and pg_restore for creating and restoring logical backups. It also supports Point-in-Time Recovery (PITR), which allows a database to be recovered to a specific time.

The purpose of this practical exercise is to create a database backup, insert new data, simulate a database failure, restore the backup, and explain how to configure PITR in PostgreSQL.

2. Taking a Full Logical Backup Using pg_dump

A logical backup is a copy of a database's structure and data that can be restored when necessary.

First, I opened the PostgreSQL command prompt and executed the following command:

```
pg_dump -U postgres -W -F c -d dvdrental -f dvdrental_backup.dump
```

The command creates a backup of the dvdrental database. The -U option specifies the username, -W requests the password, -F c selects the custom backup format, and -f specifies the backup filename.

To verify the backup, I used the following command:

```
pg_restore -l dvdrental_backup.dump
```

This command displays the database objects stored in the backup file.

Result: A logical backup of the database was created, providing a copy that could be used to restore the original database.

3. Creating a Table and Inserting Data

After creating the backup, I connected to the dvdrental database using pgAdmin or the PostgreSQL command-line interface.

I created a table named recovery_test using the following SQL statement:

```
CREATE TABLE recovery_test (
    id SERIAL PRIMARY KEY,
    name VARCHAR(50),
    department VARCHAR(50)
);
```

Next, I inserted three records into the table:

```
INSERT INTO recovery_test (name, department)
VALUES
('Thabo', 'IT'),
('Lerato', 'Networking'),
('Mpho', 'Database Administration');
```

To confirm that the records were inserted, I executed:

```
SELECT * FROM recovery_test;
```

Result: The table contained three records. This confirmed that the table had been created and the data had been inserted successfully.

4. Simulating a Failure and Restoring the Database

To simulate a database failure, I disconnected from the dvdrental database and connected to the default PostgreSQL database:

```
\c postgres
```

I then deleted the test database:

```
DROP DATABASE dvdrental;
```

This simulated a situation in which the database was accidentally deleted. I recreated the database using:

```
CREATE DATABASE dvdrental;
```

After recreating the empty database, I restored the backup using the following command in the terminal:

```
pg_restore -U postgres -W -d dvdrental dvdrental_backup.dump
```

After the restoration, I connected to the database and checked whether the test table existed:

```
\c dvdrental

SELECT * FROM recovery_test;
```

Expected result: PostgreSQL reports that the recovery_test table does not exist.

This is because the backup was taken before the table was created. Therefore, restoring the backup returned the database to its earlier state, excluding the table and records added afterwards.

To verify that the original database was restored, I executed:

```
SELECT * FROM film LIMIT 5;
```

If the original film records were displayed, it indicated that the database had been restored successfully.

Note: These steps should only be performed on a test database because dropping a database deletes its current contents.

5. Research: Setting Up Point-in-Time Recovery (PITR) in PostgreSQL

5.1 Definition of PITR

Point-in-Time Recovery (PITR) is a PostgreSQL recovery method that allows a database administrator to restore a database to a specific date and time before an unwanted change occurred. It uses a physical base backup together with archived Write-Ahead Logging (WAL) files.

For example, if a user accidentally deletes important records, PITR can help recover the database to a time before the deletion, provided the necessary backups and transaction logs are available.

5.2 Steps for Setting Up PITR

Step 1: Create a WAL archive directory.

A directory is needed to store archived transaction log files. On Linux, the following command can be used:

```
sudo mkdir -p /var/lib/postgresql/wal_archive
```

The PostgreSQL service account must have permission to write to this directory.

Step 2: Configure PostgreSQL.

Open the postgresql.conf configuration file and add the following settings:

```
wal_level = replica
archive_mode = on
archive_command = 'test ! -f /var/lib/postgresql/wal_archive/%f && cp %p
/var/lib/postgresql/wal_archive/%f'
```

These settings enable the required WAL information and copy completed WAL files to the archive directory.

Step 3: Restart PostgreSQL.

Restart the PostgreSQL service to apply the configuration changes. The archive_mode setting requires a server restart.

Step 4: Create a physical base backup.

Execute the following command:

```
pg_basebackup -U postgres -D /backup/base_backup -Fp -Xs -P
```

This command creates a physical backup of the PostgreSQL database cluster. The destination directory must be available and writable.

Step 5: Verify WAL archiving.

Execute the following SQL statement:

```
SELECT pg_switch_wal();
```

This requests a WAL segment switch. Check the archive directory and PostgreSQL logs to confirm that WAL files are being archived successfully.

Step 6: Configure recovery to a specific time.

When a failure occurs, stop PostgreSQL and restore the physical base backup into the appropriate data directory. Preserve the existing data directory first if possible.

Create a file named recovery.signal in the restored data directory and add the following settings to postgresql.conf:

```
restore_command = 'cp /var/lib/postgresql/wal_archive/%f %p'
recovery_target_time = '2026-10-21 14:25:00'
recovery_target_action = 'pause'
```

The date and time shown are examples and should be replaced with the desired recovery time. The restore_command retrieves the archived WAL files, while recovery_target_time specifies when recovery should stop.

Step 7: Start PostgreSQL and verify the recovery.

Start the PostgreSQL service and allow it to replay the archived WAL files until it reaches the selected recovery point. Check the database tables and records to confirm that the database has been recovered to the expected state.

Once the recovery has been verified, the administrator can promote the recovered database and return it to normal operation.

PITR requires a valid physical base backup and all necessary WAL files. A logical backup created with pg_dump alone is not sufficient for PITR.

6. Difference Between Logical Backup and PITR

Feature	Logical Backup	Point-in-Time Recovery
Purpose	Saves database objects and data	Recovers a database to a selected time
Tools	pg_dump and pg_restore	pg_basebackup and WAL archiving
Backup type	Logical	Physical backup with transaction logs
Recovery	Restores the saved database state	Replays changes to a selected recovery point
Example	Recovering a deleted database from a backup	Recovering data from before an accidental deletion

7. Conclusion

In conclusion, database backup and recovery are essential for maintaining data availability and preventing permanent data loss. The pg_dump utility can create logical backups, while pg_restore can restore the saved database state. The practical exercise demonstrates that changes made after a backup are not included when that earlier backup is restored.

Point-in-Time Recovery provides an additional level of protection by allowing PostgreSQL databases to be recovered to a specific time using a physical base backup

and archived WAL files. Regular backups, proper storage of backup files, and testing recovery procedures are important practices for database administrators.

2.Performance Analysis Report Using PostgreSQL

1. Introduction

Database performance analysis is an important task in database administration because it helps identify queries that take too long to execute and consume excessive system resources. PostgreSQL provides a monitoring extension called pg_stat_statements, which records statistics about SQL queries executed on the database. This report explains how to identify the three slowest queries and recommends optimization strategies to improve database performance.

2. Identifying the Three Longest-Running Queries

First, the pg_stat_statements extension must be enabled in PostgreSQL. The administrator should add pg_stat_statements to the shared_preload_libraries setting in postgresql.conf and restart the PostgreSQL server if it is not already configured. The extension can then be enabled in the target database using:

```
CREATE EXTENSION IF NOT EXISTS pg_stat_statements;
```

The following SQL query identifies the top three queries with the highest total execution time:

```
SELECT
    query,
    calls,
    ROUND(total_exec_time::numeric, 2) AS total_time_ms,
    ROUND(mean_exec_time::numeric, 2) AS average_time_ms
FROM pg_stat_statements
ORDER BY total_exec_time DESC
LIMIT 3;
```

The results show the queries that have consumed the most execution time since the statistics were last reset. The average execution time helps determine whether individual executions are slow, while the number of calls indicates how frequently each query runs.

3. Performance Findings and Recommendations

The following are three common query patterns that may cause performance problems. They are illustrative examples; actual findings must be based on the results returned by the database.

Query 1: Searching a large table without an appropriate index

A query that searches a large table using a column without an index may require PostgreSQL to scan many rows. This can increase execution time, especially when the table contains thousands or millions of records.

Recommendation: Create an index on the frequently searched column. For example, an index on `customer_id` may improve queries that filter by that column. The execution plan should be checked before and after creating the index to confirm the improvement.

Query 2: Joining several large tables

Queries that join multiple large tables may take a long time when join columns are not indexed or when the query processes more rows than necessary.

Recommendation: Create suitable indexes on frequently used join columns, select only the required columns, and apply filters as early as appropriate. Use `EXPLAIN (ANALYZE, BUFFERS)` to examine the query execution plan and identify expensive operations.

Query 3: Retrieving unnecessary data

Queries that use `SELECT *` may retrieve more columns and rows than the application requires. This can increase disk activity, memory usage, and network traffic.

Recommendation: Select only the required columns, add appropriate filtering conditions, and use pagination when displaying large result sets. These changes can reduce the amount of data processed and transferred.

4. Conclusion

PostgreSQL's `pg_stat_statements` extension provides useful information for identifying queries that consume the most execution time. Indexing, rewriting inefficient queries, and reducing unnecessary data retrieval can improve database performance. However, optimization decisions should be based on actual query statistics and execution plans. After making changes, the queries should be tested again to determine whether performance has improved.